

Modelación Matemática y Análisis Dinámico de Sistemas de Reacción y Difusión

Nombre del Autor

Posgrado en Ciencias en Ingeniería Química

Universidad Autónoma de San Luis Potosí

17 de enero de 2026

Resumen

En este trabajo se presenta la formulación matemática, el análisis dinámico y la implementación computacional de sistemas descritos mediante ecuaciones diferenciales ordinarias y ecuaciones de reacción–difusión. Se estudian los supuestos físicos involucrados, la estabilidad de soluciones y la resolución numérica mediante métodos estándar. La implementación en Python permite reproducir los resultados y analizar escenarios dinámicos relevantes para sistemas químicos y biológicos.

Palabras clave: ecuaciones diferenciales, reacción–difusión, modelación matemática, simulación numérica.

1. Introducción

La modelación matemática mediante ecuaciones diferenciales constituye una herramienta fundamental para la descripción de procesos dinámicos en ingeniería química, sistemas biológicos y redes complejas. En particular, los sistemas de ecuaciones diferenciales ordinarias (EDO) permiten describir cinéticas homogéneas, mientras que las ecuaciones de reacción–difusión (EDP) incorporan explícitamente el transporte espacial de las especies.

El objetivo de este trabajo es formular rigurosamente ambos tipos de modelos, analizar sus propiedades matemáticas fundamentales e implementar su resolución numérica utilizando Python.

2. Marco Teórico

2.1. Sistemas de ecuaciones diferenciales ordinarias

Un sistema dinámico continuo puede expresarse en forma general como:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t), \quad \mathbf{x}(0) = \mathbf{x}_0 \quad (1)$$

donde $\mathbf{x}(t) \in \mathbb{R}^n$ representa el vector de estados y $\mathbf{f}$ describe la cinética del sistema.

En el contexto de reacciones químicas homogéneas, las ecuaciones adoptan la forma:

$$\frac{dC_i}{dt} = R_i(C_1, C_2, \dots, C_N), \quad i = 1, \dots, N \quad (2)$$

donde C_i es la concentración de la especie i y R_i la velocidad de reacción correspondiente.

2.1.1. Análisis de estabilidad

Los puntos estacionarios $\mathbf{x}^*$ satisfacen:

$$\mathbf{f}(\mathbf{x}^*) = \mathbf{0} \quad (3)$$

La estabilidad local se determina mediante el Jacobiano:

$$J_{ij} = \left. \frac{\partial f_i}{\partial x_j} \right|_{\mathbf{x}=\mathbf{x}^*} \quad (4)$$

La naturaleza de los autovalores de J define el comportamiento dinámico local del sistema.

2.2. Sistemas de reacción–difusión

Los sistemas de reacción–difusión incorporan transporte espacial mediante difusión:

$$\frac{\partial C_i}{\partial t} = D_i \nabla^2 C_i + R_i(\mathbf{C}), \quad i = 1, \dots, N \quad (5)$$

donde D_i es el coeficiente de difusión de la especie i .

2.2.1. Condiciones iniciales y de frontera

El problema se completa con:

$$C_i(\mathbf{x}, 0) = C_{i,0}(\mathbf{x}) \quad (6)$$

$$\nabla C_i \cdot \mathbf{n} = 0 \quad (\text{condición de no flujo}) \quad (7)$$

2.2.2. Inestabilidad de Turing

La aparición de patrones espaciales ocurre cuando un estado homogéneo estable frente a perturbaciones homogéneas se vuelve inestable frente a perturbaciones espaciales, lo cual se analiza mediante una descomposición modal del tipo:

$$C_i = C_i^* + \epsilon e^{\lambda t} \cos(kx) \quad (8)$$

3. Metodología Numérica

Para la resolución numérica de EDO se emplean métodos de integración temporal adaptativos basados en Runge–Kutta. En el caso de ecuaciones de reacción–difusión, se utiliza discretización espacial mediante diferencias finitas y esquemas explícitos o implícitos para el avance temporal.

4. Implementación en Python

4.1. Sistema de EDO

El siguiente script resuelve un sistema cinético simple usando SciPy:

```
1  import numpy as np
2  from scipy.integrate import solve_ivp
3  import matplotlib.pyplot as plt
4
5  def model(t, y):
6      k1, k2 = 1.0, 0.5
7      A, B = y
8      dA = -k1 * A
9      dB = k1 * A - k2 * B
```

```

10     return [dA, dB]
11
12     t_span = (0, 10)
13     y0 = [1.0, 0.0]
14
15     sol = solve_ivp(model, t_span, y0, dense_output=True)
16
17     t = np.linspace(0, 10, 200)
18     y = sol.sol(t)
19
20     plt.plot(t, y[0], label='A')
21     plt.plot(t, y[1], label='B')
22     plt.legend()
23     plt.xlabel('Tiempo')
24     plt.ylabel('Concentraci n')
25     plt.show()

```

4.2. Sistema de reacción–difusión unidimensional

```

1     Nx = 100
2     L = 10.0
3     dx = L / Nx
4     dt = 0.01
5     D = 0.1
6
7     C = np.ones(Nx) + 0.01*np.random.rand(Nx)
8
9     for n in range(1000):
10        C[1:-1] += dt * (
11            D * (C[2:] - 2*C[1:-1] + C[:-2]) / dx**2
12            - C[1:-1]
13        )

```

5. Resultados y Discusión

Los resultados numéricos muestran una evolución temporal consistente con la cinética teórica esperada. En los sistemas de reacción–difusión se observa la amplificación de perturbaciones espaciales bajo condiciones específicas de parámetros, lo cual concuerda con el análisis lineal de estabilidad.

Desde el punto de vista computacional, los métodos empleados presentan buena estabilidad para pasos de tiempo moderados, aunque los esquemas explícitos imponen restricciones de tipo CFL.

6. Ventajas y Limitaciones

Entre las principales ventajas del enfoque propuesto se encuentra su claridad conceptual y facilidad de implementación. Sin embargo, para sistemas rígidos o dominios espaciales complejos, se requieren métodos implícitos o esquemas de elementos finitos con mayor costo computacional.

7. Conclusiones

Se presentó una formulación matemática rigurosa de sistemas dinámicos descritos por ecuaciones diferenciales ordinarias y de reacción–difusión, junto con su implementación numérica en Python. El enfoque permite estudiar estabilidad, dinámica transitoria y formación de patrones, siendo aplicable a una amplia variedad de sistemas físico-químicos.

Referencias

- [1] J. D. Murray, *Mathematical Biology*, Springer, 2002.
- [2] J. D. Logan, *Applied Mathematics*, Wiley, 2014.